

Prometheus on Kubernetes for LLM Inference

Monitoring, Alerting & Autoscaling for Production LLM Serving

Lecture Roadmap

1. Why observability matters for LLM inference
2. Prometheus fundamentals
3. Deploying Prometheus on Kubernetes
4. LLM inference metrics that matter
5. Instrumenting vLLM
6. Key PromQL queries
7. Grafana dashboards & alerting
8. Autoscaling with Prometheus metrics
9. Production patterns
10. Hands-on recap

Why Observability Matters for LLM Inference

LLM inference is **resource-intensive** and **latency-sensitive**:

- GPU memory is finite — KV cache pressure causes OOM or request rejection
- Token generation is sequential — one slow request blocks batch slots
- Cost scales with GPU-seconds, not just request count
- Users feel latency in time-to-first-token (TTFT), not just total duration

Without metrics, you're flying blind:

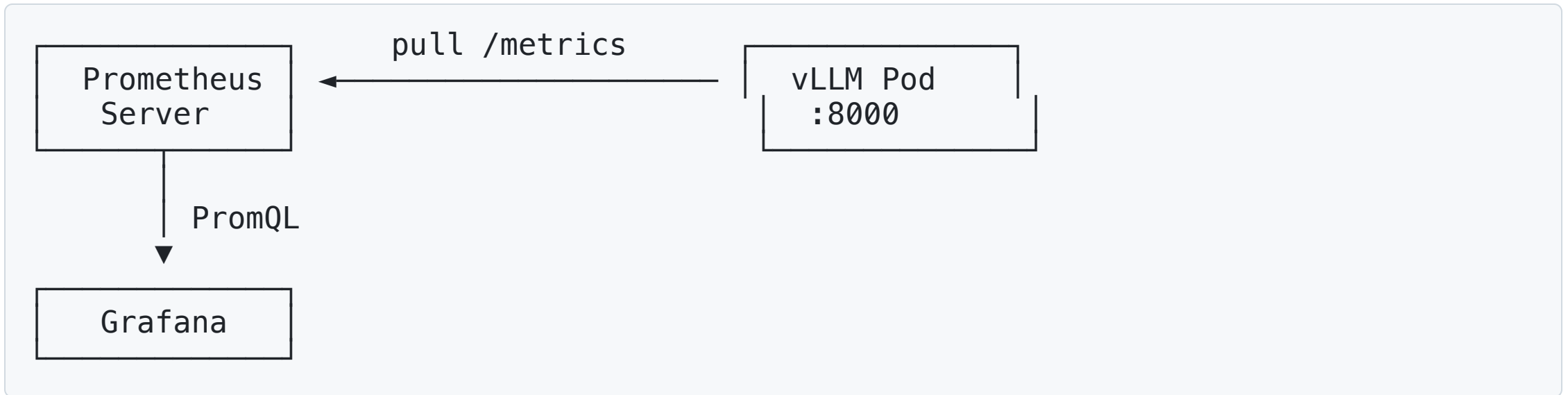
- Is the model underutilized or saturated?
- Are requests queuing or being dropped?
- What's the actual cost per token?

The Four Golden Signals for LLM Inference

Signal	Traditional	LLM Inference
Latency	Request duration	TTFT + TPOT + E2E latency
Traffic	Requests/sec	Tokens/sec (input + output)
Errors	HTTP 5xx	OOM, timeout, generation errors
Saturation	CPU/memory	KV cache %, GPU util %, batch queue depth

Prometheus Fundamentals

Architecture:



- **Pull-based model:** Prometheus scrapes targets at regular intervals
- **Time-series database:** Labels enable multi-dimensional queries
- **PromQL:** Powerful query language for aggregation and alerting

Prometheus Metric Types

Type	Use Case	Example
Counter	Monotonically increasing	<code>vllm:request_success_total</code>
Gauge	Current value (can go up/down)	<code>vllm:num_requests_running</code>
Histogram	Distribution of values	<code>vllm:e2e_request_latency_seconds</code>
Summary	Pre-calculated quantiles	<code>vllm:time_to_first_token_seconds</code>

Labels add dimensions:

```
vllm:e2e_request_latency_seconds_bucket{model_name="llama-3.2-1b", le="1.0"}
```

Deploying Prometheus on Kubernetes

kube-prometheus-stack — the standard approach:

```
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts

helm install kube-prom prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace \
  --set grafana.enabled=true \
  --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false
```

This deploys:

- Prometheus Operator + Prometheus instance
- Grafana with pre-built dashboards
- Node exporter, kube-state-metrics
- Alertmanager

ServiceMonitor: Telling Prometheus What to Scrape

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: vllm-monitor
  namespace: vllm
  labels:
    release: kube-prom
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: vllm
  endpoints:
    - port: http
      path: /metrics
      interval: 15s
  namespaceSelector:
    matchNames:
      - vllm
```

The Prometheus Operator watches for `ServiceMonitor` CRDs and auto-configures scrape targets

vLLM's Built-In Metrics Endpoint

vLLM exposes Prometheus metrics at `/metrics` by default on the API port:

```
curl http://vllm-server:8000/metrics
```

No extra instrumentation needed. vLLM tracks:

- Request counts and latencies
- Token generation throughput
- KV cache utilization
- Batch sizes and queue depths
- GPU memory usage
- Prefix cache hit rates

Key vLLM Metrics

Metric	Type	What It Tells You
<code>vllm:e2e_request_latency_seconds</code>	Histogram	End-to-end request duration
<code>vllm:time_to_first_token_seconds</code>	Histogram	Time to first token (TTFT)
<code>vllm:time_per_output_token_seconds</code>	Histogram	Inter-token latency (TPOT)
<code>vllm:num_requests_running</code>	Gauge	Active requests in engine
<code>vllm:num_requests_waiting</code>	Gauge	Queued requests
<code>vllm:gpu_cache_usage_perc</code>	Gauge	KV cache memory usage (0-1)
<code>vllm:prompt_tokens_total</code>	Counter	Total input tokens processed
<code>vllm:generation_tokens_total</code>	Counter	Total output tokens generated
<code>vllm:request_success_total</code>	Counter	Successful completions

Enabling Detailed Metrics in vLLM

In your Deployment spec:

```
containers:  
- name: vllm  
  image: vllm/vllm-openai:latest  
  args: ["--model", "meta-llama/Llama-3.2-1B-Instruct",  
        "--port", "8000", "--enable-metrics"]  
  ports:  
  - name: http  
    containerPort: 8000
```

Service Configuration for Scraping

The Service port must be **named** (required by ServiceMonitor):

```
apiVersion: v1
kind: Service
metadata:
  name: vllm-server
  namespace: vllm
  labels:
    app.kubernetes.io/name: vllm
spec:
  ports:
    - name: http
      port: 8000
      targetPort: 8000
  selector:
    app.kubernetes.io/name: vllm
```

The `app.kubernetes.io/name: vllm` label connects this Service to the `ServiceMonitor` selector.

PromQL: Request Latency Percentiles

P50, P95, P99 end-to-end latency:

```
histogram_quantile(0.95,  
  sum(rate(vllm:e2e_request_latency_seconds_bucket[5m])) by (le)  
)
```

P95 time-to-first-token:

```
histogram_quantile(0.95,  
  sum(rate(vllm:time_to_first_token_seconds_bucket[5m])) by (le)  
)
```

Interpretation:

- $TTF\text{📄} > 2s \rightarrow$ prefill is bottlenecked (long prompts, insufficient compute)
- $TPOT > 100ms \rightarrow$ decode is saturated (reduce batch size or scale up)

PromQL: Throughput & Utilization

Token throughput (tokens/sec):

```
sum(rate(vllm:generation_tokens_total[5m]))
```

Request rate:

```
sum(rate(vllm:request_success_total[5m]))
```

KV cache pressure:

```
vllm:gpu_cache_usage_perc
```

PromQL: Error Rates & Availability

Error rate:

```
sum(rate(vllm:request_failure_total[5m]))  
/  
sum(rate(vllm:request_success_total[5m]) + rate(vllm:request_failure_total[5m]))
```

Request success SLI (availability):

```
1 - (  
  sum(increase(vllm:request_failure_total[1h]))  
  /  
  sum(increase(vllm:request_success_total[1h]) + increase(vllm:request_failure_total[1h]))  
)
```

Target: 99.9% success rate over rolling 1h window.

Grafana Dashboard: Essential Panels

Overview	Resources
Requests/sec (counter rate)	KV cache usage % (alert at 85%)
Tokens/sec (input + output)	GPU utilization % (DCGM exporter)
Active vs. queued requests	GPU memory used/total

Latency	Errors
TTFT P50/P95/P99	Error rate (counter rate)
TPOT P50/P95/P99	Timeout count
E2E latency heatmap	OOM events

Alerting Rules

```
groups:
- name: vllm.rules
  rules:
  - alert: VLLMHighLatency
    expr: histogram_quantile(0.95,
      sum(rate(vllm:e2e_request_latency_seconds_bucket[5m])) by (le)) > 5
    for: 2m
    labels: { severity: warning }
    annotations:
      summary: "vLLM P95 latency > 5s"
  - alert: VLLMKVCacheSaturation
    expr: vllm:gpu_cache_usage_perc > 0.9
    for: 1m
    labels: { severity: critical }
    annotations:
      summary: "KV cache > 90% – requests will be rejected"
```

Deployed as a PrometheusRule CRD in the monitoring namespace.

More Alert Rules

```
- alert: VLLMRequestQueueing
  expr: vllm:num_requests_waiting > 10
  for: 3m
  labels: { severity: warning }
  annotations:
    summary: "Requests queueing – consider scaling up"
- alert: VLLMHighErrorRate
  expr: sum(rate(vllm:request_failure_total[5m]))
    / (sum(rate(vllm:request_success_total[5m]))
      + sum(rate(vllm:request_failure_total[5m]))) > 0.01
  for: 5m
  labels: { severity: critical }
  annotations:
    summary: "Error rate > 1%"
```

GPU Monitoring with DCGM Exporter

NVIDIA DCGM (Data Center GPU Manager) exposes GPU metrics to Prometheus:

```
helm install dcgm-exporter gpu-helm-charts/dcgm-exporter \
  --namespace monitoring \
  --set serviceMonitor.enabled=true
```

Key metrics:

Metric	Description
DCGM_FI_DEV_GPU_UTIL	GPU compute utilization %
DCGM_FI_DEV_MEM_COPY_UTIL	Memory bandwidth utilization
DCGM_FI_DEV_FB_USED	Framebuffer memory used (MB)
DCGM_FI_DEV_FB_FREE	Framebuffer memory free (MB)
DCGM_FI_DEV_GPU_TEMP	GPU temperature
DCGM_FI_DEV_POWER_USAGE	Power draw (watts)

Autoscaling with KEDA + Prometheus

KEDA (Kubernetes Event-Driven Autoscaling) scales based on Prometheus queries:

```
# ScaledObject (keda.sh/v1alpha1)
spec:
  scaleTargetRef: { name: vllm-server }
  minReplicaCount: 1
  maxReplicaCount: 4
  triggers:
  - type: prometheus
    metadata:
      serverAddress: http://kube-prom-prometheus.monitoring:9090
      query: avg(vllm:gpu_cache_usage_perc{namespace="vllm"})
      threshold: "0.75"
```

Scale up when average KV cache usage > 75%.

Autoscaling: Choosing the Right Metric

Metric	Scale Signal	Pros	Cons
KV cache %	Capacity pressure	Direct measure of saturation	GPU-specific
Queue depth	Demand exceeds capacity	Simple, universal	Spiky, needs smoothing
P95 latency	SLO violation	User-facing	Lagging indicator
Tokens/sec	Throughput ceiling	Predictable	Need to know max capacity

Recommended: KV cache % as primary, queue depth as secondary.

Important: LLM inference pods have long startup times (model loading). Scale proactively, not reactively.

Production Pattern: Multi-Model Monitoring

When serving multiple models, use labels to distinguish:

```
# Per-model latency
histogram_quantile(0.95,
  sum(rate(vllm:e2e_request_latency_seconds_bucket[5m])) by (le, model_name)
)

# Per-model throughput
sum(rate(vllm:generation_tokens_total[5m])) by (model_name)

# Per-model cache pressure
vllm:gpu_cache_usage_perc{model_name=~".*"}

```

Separate dashboards and alerts per model — a 70B model has very different latency baselines than a 7B model.

Production Pattern: Cost-Per-Token Tracking

Combine Prometheus metrics with cost data:

```
# GPU cost per output token (assuming $2.50/hr per GPU)
(2.50 / 3600) / sum(rate(vllm:generation_tokens_total[5m]))
```

Recording rule:

```
- record: vllm:cost_per_1k_output_tokens
  expr: (2.50 / 3600) / (sum(rate(vllm:generation_tokens_total[5m])) / 1000)
```

Track over time — if cost/token increases, investigate batch utilization or KV cache thrashing.

Production Pattern: Canary Deployments

Use Prometheus to compare canary vs. stable:

```
# Canary vs stable latency comparison
histogram_quantile(0.95,
  sum(rate(vllm:e2e_request_latency_seconds_bucket{version="canary"}[5m])) by (le)
)
/
histogram_quantile(0.95,
  sum(rate(vllm:e2e_request_latency_seconds_bucket{version="stable"}[5m])) by (le)
)
```

If ratio > 1.5 → canary is significantly slower than stable.

Canary: Automated Rollback Alert

Fire if canary P95 exceeds 1.5x stable for 5 minutes:

```
- alert: CanaryLatencyRegression
  expr: histogram_quantile(0.95, sum(rate(
    vllm:e2e_request_latency_seconds_bucket{version="canary"}[5m]
  )) by (le)) > 1.5 * histogram_quantile(0.95, sum(rate(
    vllm:e2e_request_latency_seconds_bucket{version="stable"}[5m]
  )) by (le))
  for: 5m
  labels: { severity: critical }
  annotations:
    summary: "Canary latency regression – trigger rollback"
```

Wire to Alertmanager → webhook → Argo Rollouts / Flagger for automated rollback.

Production Pattern: Prefix Cache Monitoring

vLLM supports prefix caching — reuse KV cache for shared prompt prefixes:

```
# Prefix cache hit rate
rate(vllm:prefix_cache_hit_total[5m])
/
(rate(vllm:prefix_cache_hit_total[5m]) + rate(vllm:prefix_cache_miss_total[5m]))
```

Why it matters:

- High hit rate → lower TTFT, better GPU efficiency
- Low hit rate → prompts are too diverse, or cache is undersized
- Track after prompt template changes — a new system prompt invalidates the entire cache

Hands-On: Full Stack on Minikube

```
# 1. Start minikube
minikube start --cpus=4 --memory=10240 --disk-size=60g

# 2. Deploy vLLM (from our existing manifests)
kubectl apply -f vllm-minikube/

# 3. Deploy monitoring stack
helm install kube-prom prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace \
  --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false

# 4. Create ServiceMonitor for vLLM
kubectl apply -f vllm-service-monitor.yaml

# 5. Access Grafana
kubectl port-forward svc/kube-prom-grafana -n monitoring 3000:80
# Login: admin / prom-operator

# 6. Access Prometheus UI
kubectl port-forward svc/kube-prom-prometheus -n monitoring 9090:9090
```

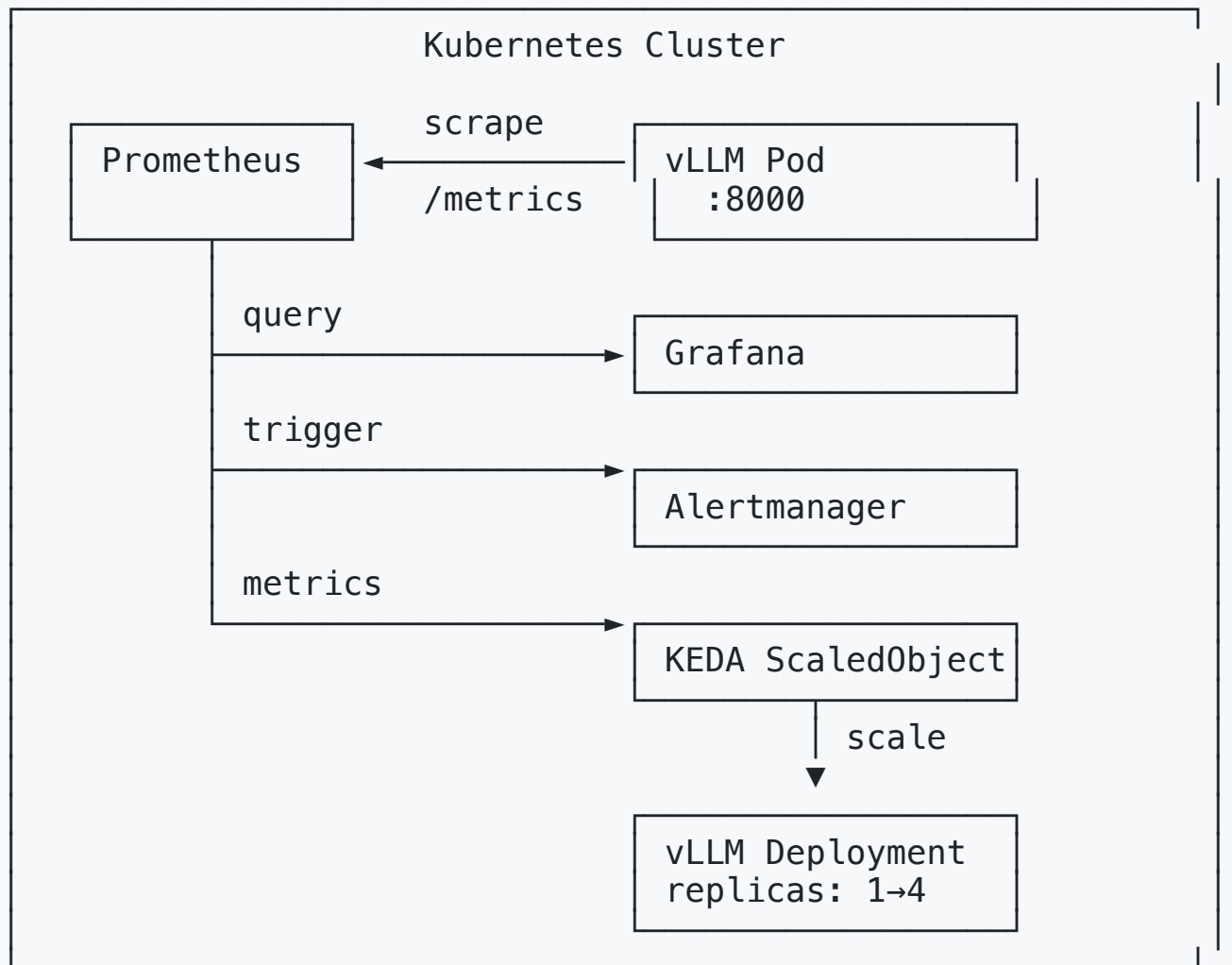
Hands-On: Verify Metrics Are Flowing

```
# Check Prometheus targets
curl -s http://localhost:9090/api/v1/targets | jq '.data.activeTargets[] |
  select(.labels.job == "vllm-monitor") | {endpoint, health}'

# Query vLLM metrics directly
curl -s http://localhost:9090/api/v1/query?query=vllm:num_requests_running

# Generate load for interesting metrics
kubectl exec -it curl-client -n vllm -- sh -c '
  for i in $(seq 1 20); do
    curl -s http://vllm-server:8000/v1/chat/completions \
      -H "Content-Type: application/json" \
      -d "{\"model\":\"meta-llama/Llama-3.2-1B-Instruct\",
        \"messages\": [{\"role\":\"user\", \"content\":\"Explain GPU $i\"}]}" &
  done
  wait
  '
```

Architecture Summary



Key Takeaways

1. **vLLM exposes Prometheus metrics natively** — no sidecar or custom code needed
2. **Use kube-prometheus-stack** — batteries-included monitoring for K8s
3. **Focus on LLM-specific signals** — TTFT, TPOT, KV cache %, queue depth
4. **Set alerts on leading indicators** — KV cache > 85% fires before users see latency
5. **Autoscale proactively** — model loading is slow; scale on capacity pressure, not SLO violations
6. **Track cost-per-token** — the metric that ties engineering to business

Further Reading

- [vLLM Metrics Documentation](#)
- [Prometheus Operator / kube-prometheus-stack](#)
- [KEDA Prometheus Scaler](#)
- [NVIDIA DCGM Exporter](#)
- [SRE Book: Monitoring Distributed Systems](#)
- [CNCF: Running Self-Hosted LLM in Kubernetes with vLLM](#)

Questions?

Try it yourself:

```
git clone <your-repo> && cd vllm-minikube  
# Follow the hands-on steps from slide 24
```